HRIS Support Ticket Tracker

A serverless ticketing API on AWS (API Gateway, Lambda, DynamoDB)

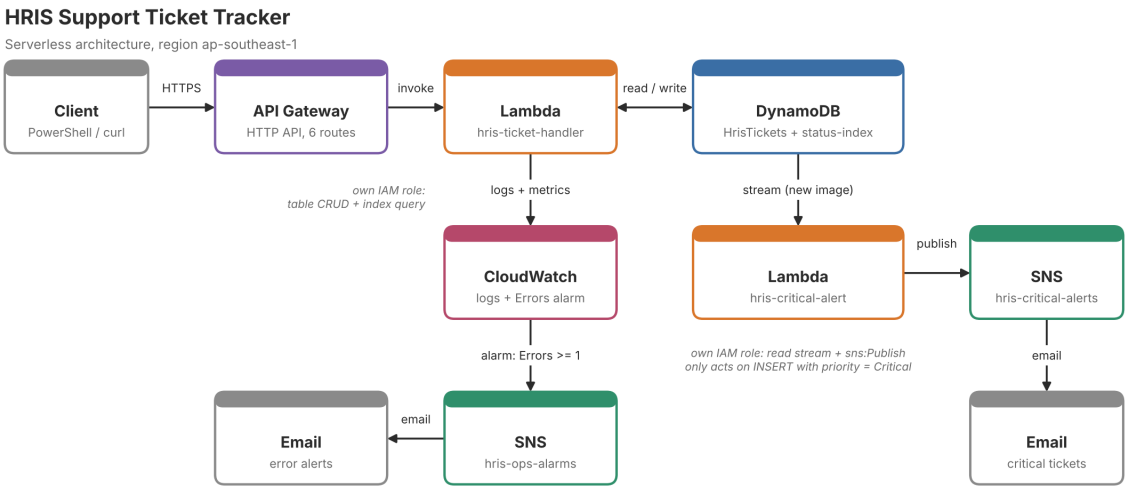
Jervil Angelo De Leon | October 20, 2026 | Region: ap-southeast-1 (Singapore)

1. Overview

This is a small ticketing API I built on AWS to practice connecting services through Lambda. It stores HRIS support tickets, the same kind of requests I handle in my day job in HRIS technical support. You can create, update, filter and delete tickets, get a summary of what is open and where, and receive an email whenever a Critical ticket is logged. All the data in it is made up.

There are no servers to manage. API Gateway receives each request, a Lambda function runs the logic, and DynamoDB stores the tickets. A second Lambda function, triggered by the table's stream, takes care of the alerts.

2. Architecture



A request goes through API Gateway to the **hris-ticket-handler** function, which reads or writes the **HrisTickets** table. Every new or changed item also appears on the table's DynamoDB stream. The **hris-critical-alert** function reads that stream, ignores everything except newly created tickets with priority Critical, and publishes those to the **hris-critical-alerts** SNS topic, which emails me. Separately, CloudWatch keeps the handler's logs and runs an alarm on its error count, which notifies me through a second SNS topic.

3. API endpoints

Method	Route	Purpose	Success
POST	`/tickets`	Create a ticket	201
GET	`/tickets`	List tickets; optional ?status= and ?module= filters	200
GET	`/tickets/{id}`	Get one ticket	200
PUT	`/tickets/{id}`	Update status, assignee, priority or resolution notes	200
DELETE	`/tickets/{id}`	Delete a ticket	200
GET	`/summary`	Counts of tickets by status and by module	200

Bad input returns 400 with a list of what is wrong, an unknown ticket returns 404, and anything unexpected returns a generic 500 while the details go to the logs. Updates and deletes use a condition check, so changing a ticket that does not exist gives a 404 instead of quietly creating or ignoring it.

4. Data model

One DynamoDB table, **HrisTickets**, with on-demand capacity. The partition key is **ticketId** (for example HRIS-70B503C7). No sort key.

Field	Notes
`customerName`	Required
`module`	Core HR, Payroll, Time & Attendance, Recruitment, Performance, Learning, Reports, Other
`requestType`	Access/Permission, Issue/Bug, How-To, Data Correction, Enhancement
`priority`	Low, Medium, High, Critical (defaults to Medium)
`status`	Open, In Progress, Waiting on Customer, Resolved, Closed (new tickets start as Open)
`assignee`	Defaults to Unassigned
`description, resolutionNotes`	Free text
`createdAt, updatedAt, dueAt, resolvedAt`	ISO 8601 timestamps in UTC; resolvedAt is only set when a ticket is resolved

There is also a global secondary index, **status-index**, with status as the partition key and createdAt as the sort key. Filtering by status uses a query on this index, so it only reads the tickets it needs and returns the newest first. Without it, the same filter would mean scanning the whole table.

The due date is set when a ticket is created, based on its priority:

Priority	Due after
Critical	4 hours
High	8 hours
Medium	24 hours
Low	72 hours

5. Security and permissions

Each Lambda function has its own execution role, and neither policy uses a wildcard on the main resource. Both roles also carry the basic CloudWatch Logs permissions that Lambda adds by default.

Function	Allowed	Only on
hris-ticket-handler	PutItem, GetItem, UpdateItem, DeleteItem, Scan; Query on the index	HrisTickets table and its status-index
hris-critical-alert	GetRecords, GetShardIterator, DescribeStream, ListStreams; sns:Publish	HrisTickets stream; hris-critical-alerts topic

The API stage is throttled to 5 requests per second with a burst of 10, mainly so a runaway script cannot cause a surprise bill. The API has **no authentication**. That was acceptable for a short-lived test with fake data, but a real deployment would need one, for example a Cognito JWT authorizer.

6. Logging and monitoring

The handler writes a JSON log line for each request with the route and the Lambda request ID, and logs error details when something fails. Log retention is set to one week on both functions so old logs do not pile up. A CloudWatch alarm watches the Errors metric of the main function and sends an email through SNS if there is at least one error in a five-minute period. Missing data is treated as good, since Lambda reports nothing when it is not being called.

7. Critical ticket alerts

DynamoDB Streams is enabled on the table with the new image view type. The trigger on **hris-critical-alert** uses a batch size of 10 and starts from the latest record, so existing tickets do not produce alerts. For each record the function checks that the event is an INSERT and that the priority is Critical. If so, it publishes the ticket ID, customer, module, request type, due time and description to the SNS topic. Anything else is skipped.

8. Testing

I tested everything from PowerShell with Invoke-RestMethod. Results:

Test	What I did	Result
Create	POST a valid ticket	201, ticket returned with an ID and a due date
Read	List all tickets; get one by ID	Stored tickets returned
Filter	?status=Open and ?status=Resolved	Correct tickets returned via the index
Update	PUT status Resolved with notes	Status changed, resolvedAt added
Summary	GET /summary	Counts by status and by module
Validation	POST with missing fields and a bad module	400 with a list of problems
Not found	GET an ID that does not exist	404
Delete	DELETE a ticket, then GET it	Deleted, then 404
Critical alert	Create a Critical ticket	Email received in about a minute
Non-critical	Create a Low ticket	No email, as intended

9. Things that went wrong

Wrong region. I started creating resources in the wrong region and had to remove them and begin again in Singapore. Worth checking the region selector before every step, because almost everything in AWS is regional.

Empty ticket ID. During testing, two commands got pasted onto one line and the ticket ID variable ended up empty. The GET and PUT calls came back as a plain 500 "internal server error", because DynamoDB does not accept an empty key. The response body does not say that, which is why the structured logs matter. Once the variable was set properly, both calls worked.

10. Limitations and next steps

- No authentication. A Cognito JWT authorizer or API keys would be the first fix.
- Listing without a status filter uses a Scan capped at 50 items. A real version needs pagination.
- Alerts fire only when a ticket is created. Raising an existing ticket to Critical through PUT sends nothing; handling MODIFY events on the stream would cover that.
- Timestamps are stored and emailed in UTC. Converting to local time would make the alerts easier to read.
- Everything was built by hand in the console. Rebuilding it as an AWS SAM template would make it repeatable with one command.
- A small frontend page would make it easier to demonstrate.

Cost. Lambda, API Gateway and DynamoDB were all used on pay-per-use pricing, and the traffic during testing was tiny, so the cost was negligible.